

BACK TO THE FUTURE

RETHINKING TIME TRAVEL
IN GIT WITH JUJUTSU



BEARDED TECH GUY WITH QUESTIONABLE FACIAL HAIR

DAD

- Foster growth
- Teach & learn everyday
- Nonviolent communication

DEVELOPER

Empowered by software craftsmanship practices

ARCHITECT

- Use industry recognized norms & practices
- Make it easy to do right, hard to do wrong



Alexandre Trigueros - CROSS by Micropol...

Je mets la technique au service du business pour aider les organisations à gagner en... · Experience: CROSS by Micropole · Education: Le Cercle Tec...

PASSIONATE

- Curious
- Actively seek improvement
- Love sharing

PRAGMATIC & INTROSPECTIVE

- Learned from how to keep the good and avoid the bad... but mostly from how to avoid the bad
- Try to avoid cargo cult
- Today's truth is not tomorrow's truth

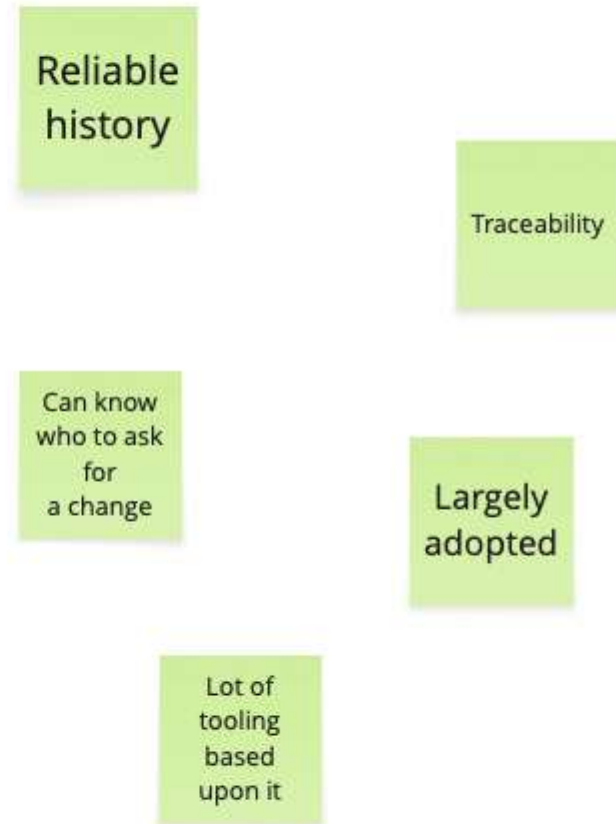
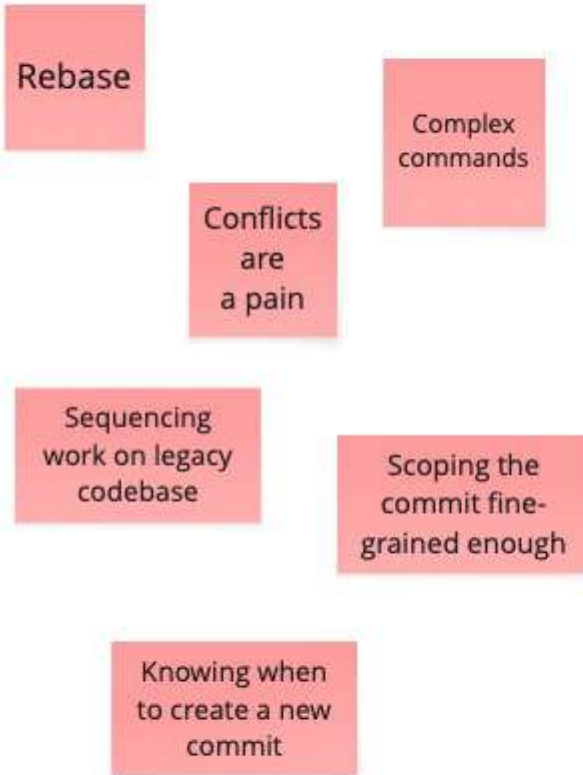
AGILIST

- Focus on business value
- Estimates are a lie
- Technical-to-business translator

Pain

Your Git experience

Gain



Martin Von Zweigbergk



- Former git maintainer
- Ex-google

Frustrated by git complexity

JUJUTSU

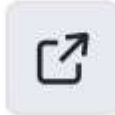


*A modern git overlay
to update history reliably, safely and easily*

Mercurial, git and **"mutable but safe history"** based

Wished to make it easy to have a clean history

INSTALLATION



Installation and setup - Jujutsu docs

There are pre-built binaries of the last released version of jj for Windows, Mac, or Linux (the "musl" version should work on all distributions).

```
brew install jj
```

```
cargo install --locked --bin jj jj-cli
```

```
winget install jj-vcs.jj
```

```
scoop install main/jj
```

CAVEAT

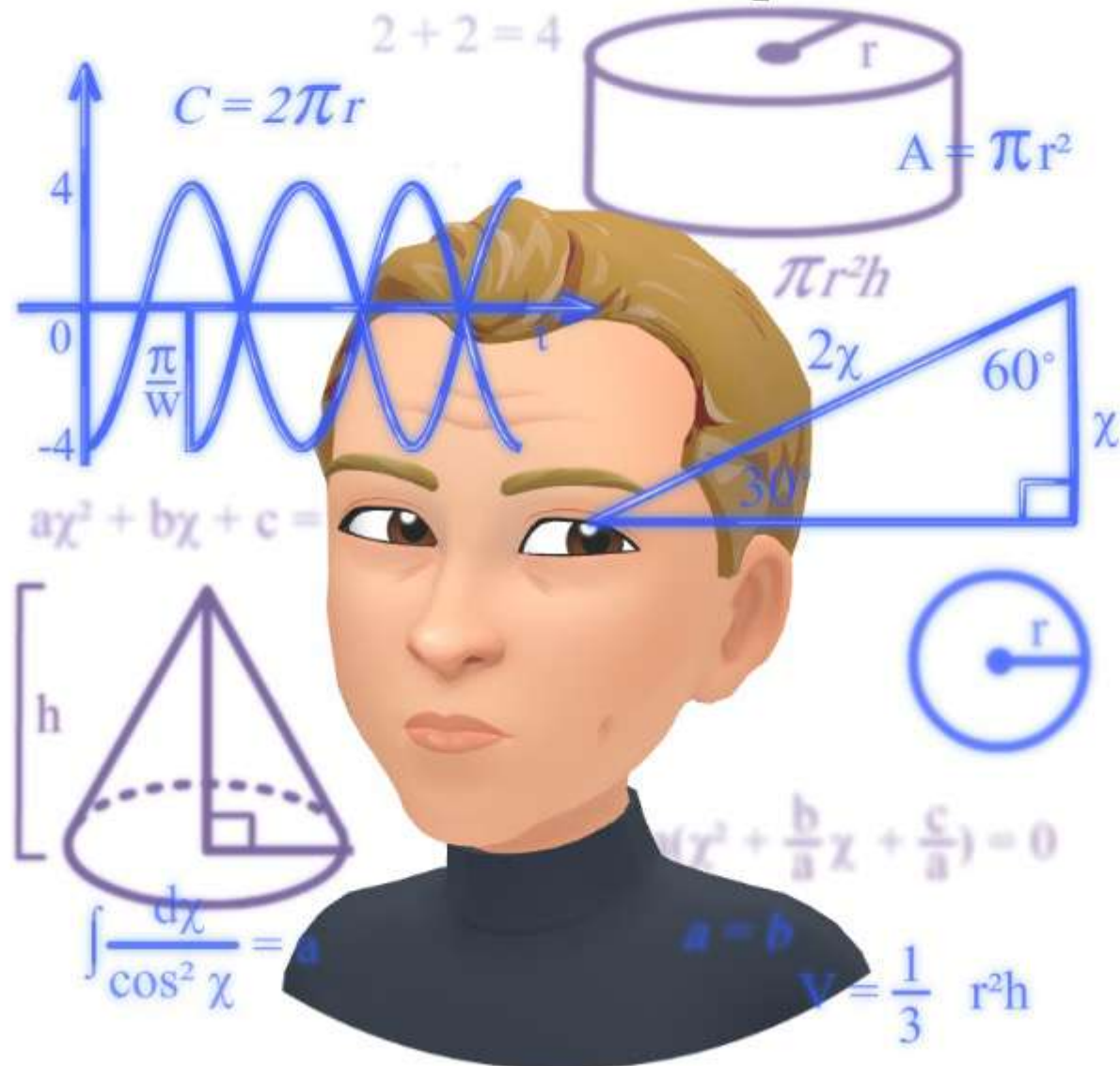
- No git module support
- Don't care about *.gitattributes* file
- Line ending normalization on windows



Working on Windows - Jujutsu docs

Jujutsu works the same on all platforms, but there are some caveats that Windows users should be aware of. Jujutsu currently has a setting, wor...

Core concepts



Working copy

No Staging area



All updates are
automatically
tracked



```
jj st  
jj status  
jj diff  
jj restore
```

```
21:13:57 > echo "hello" > world
```

```
21:14:01 > jj st  
Working copy changes:  
A world  
Working copy (@) : umqzvvr df0efed8 (no description set)  
Parent commit (@-): zzzzzzzz 00000000 (empty) (no description set)
```

Change

A change is **not** a commit



a change evolve
over time.

It... changes

Change = what you're doing

Commit = state of a change



```
jj new  
jj new {change_id}  
jj log  
jj desc  
jj describe
```

```
🕒 21:14:01 > jj st
Working copy changes:
A world
Working copy (@) : umqzvvr df0efed8 (no description set)
Parent commit (@-): zzzzzzzz 00000000 (empty) (no description set)
```

```
🕒 21:14:06 > echo "something" > "A New"
```

```
🕒 21:20:12 > jj st
Working copy changes:
A A New
A world
Working copy (@) : umqzvvr ebafb471 (no description set)
Parent commit (@-): zzzzzzzz 00000000 (empty) (no description set)
```

Commit

snapshot of a
change at a
given time

automatically managed

Not used on
a day to day basis



Conflict

Valid state,
stored in a
change

Resolved later

Resolution is a separate,
tracked step



Operation log

Every change is tracked, including rebase, conflict resolution, squash, split, etc.



jj operation log
jj undo

```
21:36:54 > jj op log
bb5a51b979e7 atrigueros@CP250002 16 minutes ago, lasted 20 milliseconds
snapshot working copy
args: jj status
b9e60394c267 atrigueros@CP250002 22 minutes ago, lasted 15 milliseconds
snapshot working copy
args: jj status
676fd61b723f atrigueros@CP250002 2 hours ago, lasted 19 milliseconds
add workspace 'default'
00000000000000 root()
```

Local manipulations



Initialize repository



Initialize a repository using ``jj git init --colocate`` command

uses ``jj log`` to see the init commit, create a file with some content, then uses ``jj st`` to see what contains the change.

Log



See the output result from the ``jj log`` command.

You can use the `--limit` argument to reduce the number of commits shown.

Status



See how the ``jj st`` command show you the modifications.

Inspect the detail using the ``jj diff`` command.

Use the ``jj diff filename`` command to limit its scope.

Describe

See how ``jj desc`` opens an editor and asks you to set a description.

Then, on another change, uses the ``jj desc -m "message"`` to directly set the message.

You can also create a new change using the ``-m "message"`` arguments.





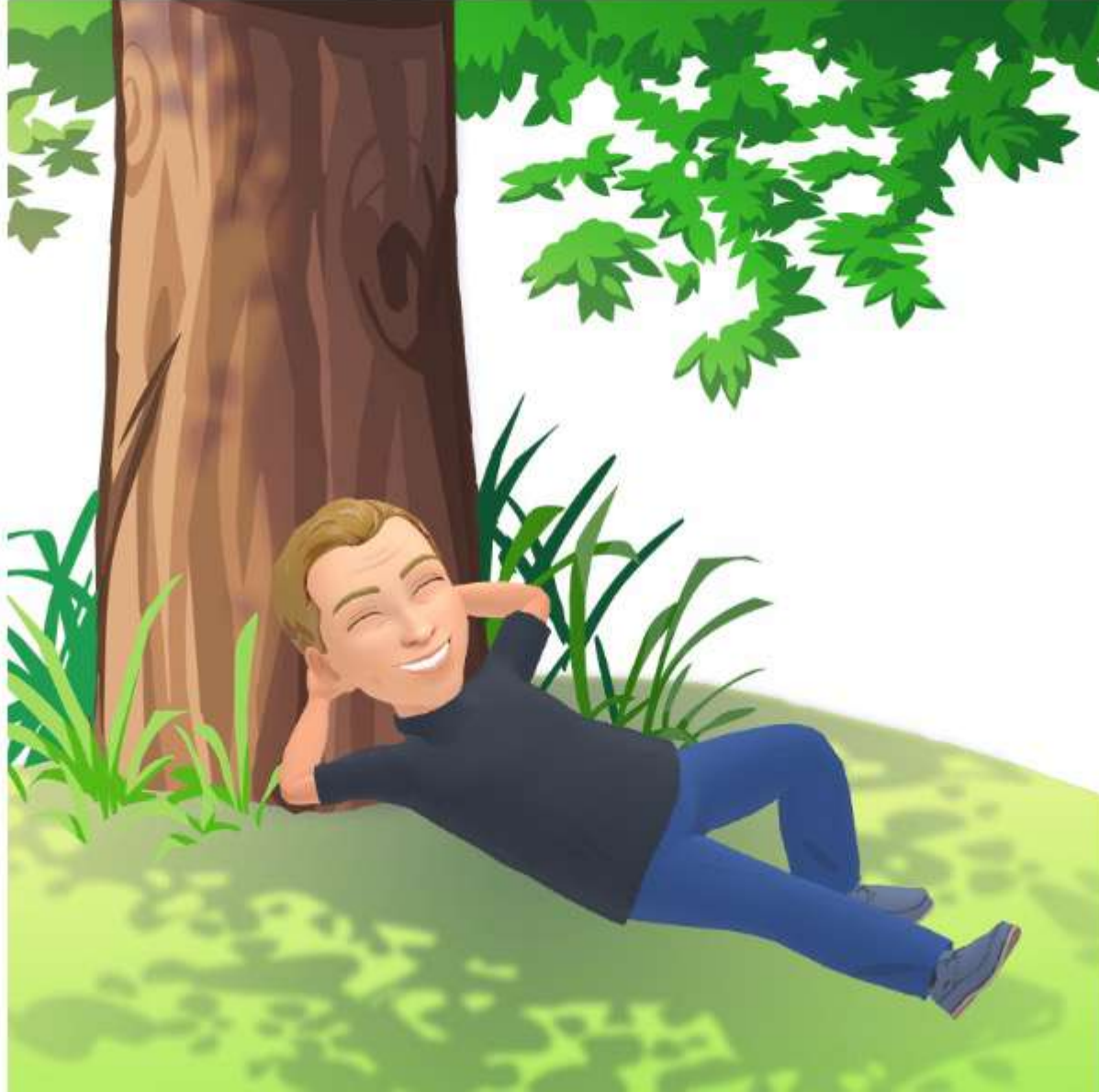
New change

Add a new change using the ``jj new`` command.

Perform some changes and create another one.

Move back in time using the ``jj edit change_id`` command and perform other modifications in the past.

Branching



Create new

uses ``jj new change_id`` on an older change id to create a new branch.

Perform some changes and uses the ``jj log`` command to see how the change history looks like.



Rebase



Develop different branches
and then rebase using the
``jj rebase --after change_id``
or ``jj rebase -s
change_id_to_move -d
change_id_to_move_after``

Conflicts

Introduce a conflict either using different branches or by rewriting the past.
See how the commit impact the change history.
Then edit the change where it happened and resolve it using the ``jj resolve`` command.



Undo



You made a mistake doing the rebase. Uses the ``jj undo`` command and see how the system reverted to the previous state/

Merge

A merge is a change over at least two branches.

Create some branches (as much as you want) and merge them using the ``jj new b1_change_id b2_change_id etc``

Notice that you are not limited to two branches.



COOL BEANS



Squash

Create two consecutive new changes.

Update some files and then uses the ``jj squash -i`` command.

Uses the arrows to dive in the changes and once you have selected what you want to move confirm using 'c'.

See how your selected edits moved to the former change.



Split

Create a new change and edit two things. Then, uses the ``jj split -i`` command.

Choose the edits you want to create in a previous intermediate change.

Confirm with 'c' and see how a new change was created.



Remote



Git model

Branch:

- Pointer towards latest commit
- Each new commit update the branch`
- HEAD = track the active branch

Jujutsu model

Bookmark:

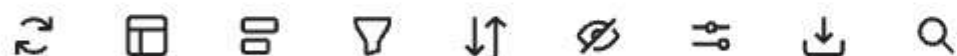
- Pointer towards a given commit
- Can be pushed / tracked as a git branch
- Manually, locally managed

Summary



↺ 📁 📂 🔍 ↕ 🔗 🔧 📄 🔍			
	📖 Concept	≡ Git	≡ Jujutsu +
1	Commit	Snapshot	Snapshot
2	HEAD	Active branch	@ (current revision)
3	Branche	Automatic pointer	Bookmark manuel
4	Remote	origin/main	main@origin
+			

Commands



🔖 Command

☰ Description

- | | | |
|---|---|---|
| 1 | <code>jj git fetch</code> | Update from remote repository |
| 2 | <code>jj git push</code> | Push changes to the remote repository |
| 3 | <code>jj git push --allow-new</code> | Push changes to the remote repository and create branches if required |
| 4 | <code>jj bookmark list</code> | List all local bookmarks |
| 5 | <code>jj bookmark list --all-remotes</code> | List all bookmarks including the ones from various remotes |

+

Commands



🔖 Command

≡ Description

1	<code>jj bookmark create my_bookmark</code>	Create a local bookmark
2	<code>jj bookmark delete my_bookmark</code>	Delete a local bookmark and propagate its deletion to the server
3	<code>jj bookmark forget my_bookmark</code>	Delete a local bookmark without propagating its deletion to the remote
4	<code>jj bookmark set my_bookmark</code>	Create or update a bookmark to point to a certain commit
5	<code>jj bookmark track my_bookmark@origin</code>	Create a local bookmark to track a remote one

More stuff



The squash workflow



- Very similar to git amend
- Create an empty change, describe what you want in it.
- Create another empty change
- Squash your edits as they fit in the former change

The edit workflow



- Very flexible
- Try to follow your description
- When it doesn't fit, split you changes
- Repeat

More content



- JJ documentation: <https://jj-vcs.github.io/jj/latest/>
- Steve Klabnik tutorial: <https://steveklabnik.github.io/jujutsu-tutorial/>
- Devops Toolbox introduction video: <https://www.youtube.com/watch?v=cZqFaMlufDY>